Framework v2: requirements-driven explanation adaptation

Oct 11, 2026 · @Minidu

Contribution

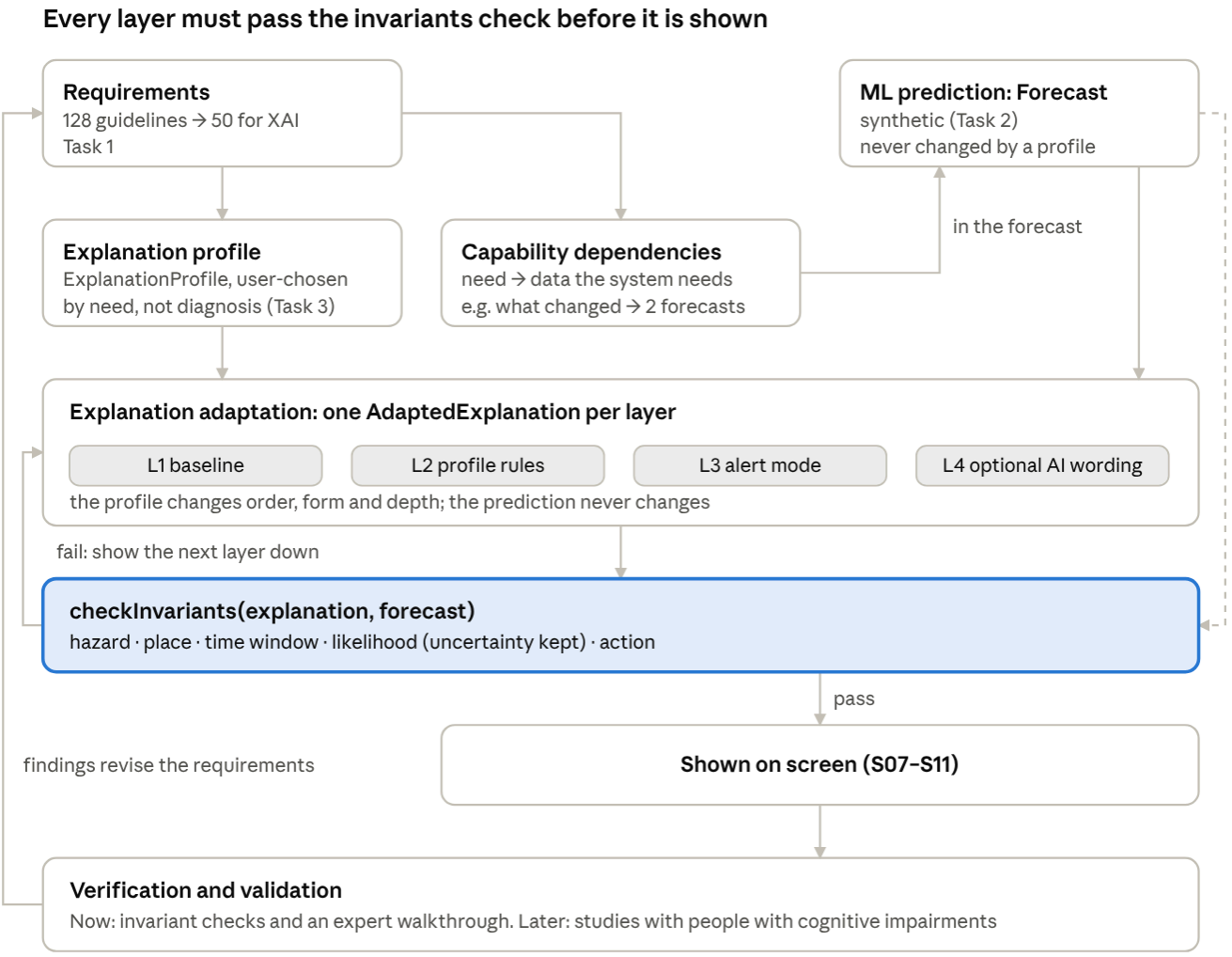
The framework turns cognitive accessibility requirements into explanation behaviour for an ML flood prediction, and checks every adapted explanation against five fidelity invariants at run time, before anyone sees it. The prediction never changes; only its explanation does.

After the related-work check (9–11 Oct), the paper claims two things as ours:

1. **Run-time fidelity invariants.** Hazard, place, time window, likelihood and action are checked on every adapted explanation, including AI-generated wording, against the forecast it explains. Uncertainty is one of the invariants, so no profile can drop it. NeuroAdaptX checks informational equivalence once, before deployment, on fixed templates, with four units that do not include uncertainty (Bunde 2026, p. 215).
2. **Requirement → capability dependencies.** Each explanation need creates data and functions the system must have, traced to fields in the data model. This goes beyond tracing requirements to interface features, which NeuroAdaptX and Madugalla already do.

We do not claim user-chosen adaptation (Madugalla et al. already offer it) or requirement-to-feature tracing as new. Both are part of the framework, cited as prior work.

Framework figure



Framework v2 · requirements and prediction meet in the layers; every layer is checked

Requirements flow down to a profile and to capabilities the forecast must provide. The profile and the forecast meet in the layers, and each layer's output is compared with the forecast (dashed line) before it is shown. Verification findings feed back into the requirements.

Fidelity invariants

No profile, simplification, alert mode or AI rewording may remove or change these five facts. Each one is a slot in `AdaptedExplanation`; `checkInvariants()` compares it with the `Forecast`.

Invariant	Example (Hanwella)	Grounding	How it is checked
Hazard	Flooding	G11, G119	Slot present; value equals `forecast.hazard`
Place	"here" (Hanwella gauge, Kelani Ganga)	G73	Slot present; gauge id equals `forecast.place.gaugeId`
Time window	on Thursday	G73, G42	Slot present; start and end equal `forecast.window`
Likelihood (uncertainty kept)	"Possible", plus 3 of 10 houses or 29% per profile	G26, G119	Slot present; CAP word equals `forecast.likelihood.capCertainty`; probability equals the forecast's. For AI wording, the CAP word must also appear in the text
Action	Get your go-bag ready today	G11, G50	Slot present; steps equal `forecast.action.steps`

Result: pass, or fail with two lists, *missing* (slot absent or text empty) and *wrong* (value differs from the forecast). Any fail means the screen shows the next layer down instead (see The four layers).

Requirement → capability dependencies

An explanation need is not only a UI choice: it requires data and functions in the system. The last column shows where each capability lives in the data model, so every row is checkable in code.

Explanation need	Guidelines	Capability the system needs	Where it lives	Status
What changed since last time	G89, G54	Stored previous forecasts and a comparison	`ForecastFile` (two forecasts per gauge); `compareForecasts()`	In the model
Why is the risk high	G25, G14	Model exposes supporting observations, never feature importance as cause	`Forecast.supportingObservations`	In the model; values from Supun

Explanation need	Guidelines	Capability the system needs	Where it lives	Status
How sure	G26, G119, G81	Probability or range from the model	`Forecast.likelihood.probability`, `Forecast.peakLevel`	In the model
Level against a landmark	G38, G76	Local reference levels for each area	`Forecast.landmark`	Placeholder; no real levels exist
When is the next update	G42	Forecast schedule metadata	`Forecast.nextUpdateAt`	In the model
What to do	G11, G50, G22	Action steps per likelihood level, from a cited source	`Forecast.action` (`source`, `placeholder`)	"Possible" only; other levels placeholder
Carer explanation	G107, G80	Consent record and a second profile	`ExplanationProfile.carerConsent`, `answeredBy`	Consent in the model; second profile not yet

Two rows are placeholders, and the paper says so: no source gives real landmark levels, and action wording is sourced only for "Possible".

The four layers

Every layer produces an `AdaptedExplanation`, and every layer's output must pass `checkInvariants()`; a failed layer falls back to the one below it.

Layer	What it does	Checked?	If the check fails
L1 Baseline	One fixed accessible explanation containing all five invariants	Yes (a fail is a build error, caught by tests)	Nothing below it: the app must not ship
L2 Profile rules	Deterministic rules choose content, order, form and depth from the profile; auditable	Yes	Show L1

Layer	What it does	Checked?	If the check fails
L3 Alert mode	When an alert is live, every profile collapses to one short line with all five facts, e.g. "Flooding possible here on Thursday. Get your go-bag ready today." (G124)	Yes	Show L1
L4 Optional AI wording	LLM rewords the L2 text	Yes, including the CAP word in the text	Show L2

Change from v1: alert mode was "action only", which would drop hazard, place, time window and likelihood and break the framework's own rule. It now shortens the wording but keeps all five facts. The line stays as short as an action-only message would be. Check this against the G124 wording in the catalogue before the meeting.

Worked trace: "How sure, and what changed?" profile

One requirement, followed from the guideline to the screen. All values are synthetic; the trace matches `model/__fixtures__/hanwella.ts`.

1. **Requirement** G26: provide alternatives for numerical concepts.
2. **Explanation need:** understand likelihood without relying on numeracy.
3. **Profile attribute:** `likelihoodForm = 'wordPictureNumber'` (preset `howSure`).
4. **Prediction:** today p = 0.29, `ABOVE_NORMAL`; yesterday p = 0.10, `NO_FLOODING`. `capCertaintyFor()` gives "Possible" for both.
5. **Capability:** `compareForecasts()` gives class changed, direction up, 1 → 3 of 10 houses.
6. **L2 rule:** CAP word "Possible" + icon array 3 of 10 + "29%" on tap + change note "Yesterday: 1 in 10".
7. **`checkInvariants()`:** hazard, place, time window, likelihood and action present and matching the forecast → pass.
8. **Shown on S09;** the requirement traces to that screen element.

Because both days are "Possible", the word alone hides the change from 10% to 29%. The trace shows why this profile keeps the picture and the number.

What changed from v1, and questions for 15 Oct

Five changes from Task 5 in Research Approach v3.1:

#	v1 (7 Oct)	v2 (11 Oct)	Why
1	Novelty included user-chosen profiles and requirement tracing	Novelty is run-time invariants (with uncertainty) and capability dependencies	Madugalla and NeuroAdaptX already do the first two
2	Alert mode: action only	Alert mode: one short line with all five facts	Action-only broke the invariant rule
3	One check box in the figure	Check after every layer, with fallback L4 → L2 → L1	Shows exactly when the check runs
4	Capability table: need → capability	Adds guidelines, data model field and status	Makes each row checkable in code; flags the two placeholders
5	Figure named no types	Figure names `Forecast`, `ExplanationProfile`, `AdaptedExplanation`	Figure matches the code

Questions for the supervisor meeting:

- [ ] Is the main SE contribution the invariants, with capability dependencies second? Or present both equally?
- [ ] Is the alert-mode change acceptable, or should alert mode check only hazard, place and action?
- [ ] Should L4 (AI wording) be built for the walkthrough, or described only as a design?

Next: replace Task 5 in the Research Approach with this text, then check the 50 guidelines against it (38 Direct first).